Cabin Booking System

Security & Compliance Dossier

Audience: Security & Compliance Manager • InfoSec / Data Protection reviewers

Deployment: Frontend + API on **Vercel** • Database on **Supabase (Postgres)** • TLS enforced (HSTS preload)

Scope: Internal room booking for ~200 employees (@**codeace.com**) • Minimal PII • No secrets in repository

Prepared: 2026-09-22 • Version 1.0 • Source: direct read of `frontend/lib/bookingApi.js` & `supabase/schema.sql`

Property	Value
Users	~200 employees — 3 roles: ADMIN / TEAM_LEAD / EMPLOYEE
Hosting — App + API	Vercel (Vite SPA + Node serverless `frontend/api/index.js:1`)
Hosting — Database	Supabase Postgres (`supabase/schema.sql:1`)
Auth	Google OAuth 2.0 / Google Identity Services (@react-oauth/google)
TLS	HTTPS only — HSTS max-age=63072000; includeSubDomains; preload
PII scope	Minimal: work email, display name, department, booking purpose

> **Audience:** Security & Compliance Manager, InfoSec / Data Protection reviewers **System:** Internal cabin/room booking for ~200 employees (`@codeace.com`) **Current deployment:** Frontend + API on **Vercel**, Database on **Supabase (Postgres) Prepared:** 2026-09-22 | **Version:** 1.0 | **Classification:** Internal

Executive Summary

Cabin Booking is a single-page internal web app that lets authenticated employees view cabin availability and reserve meeting rooms in 30-minute slots (09:00–19:00 IST). It is **not a public product**, not indexed by search engines, and requires a pre-provisioned company Google account.

Property	Value
Users	~200 employees, 3 roles (ADMIN / TEAM_LEAD / EMPLOYEE)
Hosting — app + API	**Vercel** (Vite SPA + Node serverless `frontend/api/index.js:1`)
Hosting — database	**Supabase Postgres** (`supabase/schema.sql:1`)
Auth provider	**Google OAuth 2.0 / Google Identity Services** (`@react-oauth/google`)
Data residency	Supabase Cloud (managed Postgres, region per Supabase project settings)
TLS	Enforced — HSTS `max-age=63072000; includeSubDomains; preload` (`frontend/vercel.json:28`)
PII scope	Minimal: work email, display name, department, booking purpose
Secrets in repo	**None committed** — `.env.example` only placeholders (`frontend/.env.example:1`)

System Design

2.1 Architecture

2.2 Technology Stack

Layer	Technology	Notes
Frontend	React 19.2, TypeScript ~6.0, Vite 8.3, React Router 7.18, Tailwind 4.3	SPA (`frontend/src/App.tsx:1`)
Auth client	`@react-oauth/google` 0.13.5, `@supabase/supabase-js` 2.116	`frontend/src/api/appsScript.ts:1`
Backend API	Node.js serverless on Vercel, `frontend/lib/bookingApi.js:1`	Single handler, 18 actions
Database	Supabase Postgres + pgcrypto	`supabase/schema.sql:3`
Slack (optional)	Bot Token + Signing Secret	`frontend/lib/bookingApi.js:288`
Build/Deploy	`vite build` → `dist`, `vercel.json:2` headers + rewrites	

2.3 Request Flow (Happy Path — Book a Cabin)

1. User clicks **Continue with Google** (`frontend/src/App.tsx:20` — `useGoogleLogin` with `scope: 'openid email profile'` and `hosted_domain: 'codeace.com'`).
2. Google returns `access_token` (60 min lifetime). Client stores it **in-memory + `sessionStorage` with 55 min TTL** (`frontend/src/api/appsScript.ts:22`) and calls `POST /api { action: 'currentUser', accessToken }` with `credentials:'include'` (`frontend/src/api/appsScript.ts:149`).
3. Server verifies token (`frontend/lib/bookingApi.js:712` — see §4), looks up `users` row, rejects if not found or `active=false` (`frontend/lib/bookingApi.js:800`).
4. Server sets **HttpOnly `__Host-session` cookie** `SameSite=Lax; Secure; Max-Age=3300` (`frontend/lib/bookingApi.js:943`) so subsequent requests can authenticate via cookie as fallback (`frontend/lib/bookingApi.js:969`).
5. Create lock → `POST /api { action:'createLock', cabinId, date, startTime, endTime }` → `supabase.rpc('create_booking_lock',…)` with `pg_advisory_xact_lock(hashtext(cabin:date))` (`supabase/schema.sql:170`), overlap checks, 5-min expiry.
6. Confirm → `POST /api { action:'confirmBooking', lockId, purpose, attendeeEmails }` → `confirm_booking_from_lock` RPC (`supabase/schema.sql:210`) re-locks, re-validates, inserts `bookings` row, deletes lock, writes notifications + Slack DMs + audit log.

2.4 Environments

Env	Frontend URL	API	DB
Production	`https://cabin.codeace.org` (primary, `vercel.json:82`) — also allows `cabin.codeace.com` + `*.codeace.org/com` subdomains (`lib/bookingApi.js:1009`)	Same origin `/api`	Supabase project via `SUPABASE_URL`
Local dev	`vite` dev server with `bookingApiPlugin` (`frontend/vite.config.ts:5`)	Mounted middleware at `/api`	Same Supabase (service key from env)

Hosting & Infrastructure

3.1 Vercel (Application Hosting)

- **What runs there:** Static SPA (`dist`) + serverless functions (`frontend/api/index.js:1`, `frontend/api/slack/events.js:1` via `frontend/lib/bookingApi.js:998`).
- **TLS:** All traffic HTTPS. HSTS `63072000; includeSubDomains; preload` on every response (`frontend/vercel.json:28` + `lib/bookingApi.js:983`).
- **Headers (global, `vercel.json:2`):** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: camera=(), microphone=(), geolocation=()`, `Strict-Transport-Security`, full `Content-Security-Policy` (`vercel.json:32`), `Cross-Origin-Opener-Policy: same-origin-allow-popups`, `Cross-Origin-Embedder-Policy: credentialless`, `Cross-Origin-Resource-Policy: same-origin`. API variant adds `Cache-Control: no-store` and `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'` (`vercel.json:48`).
- **SPA routing:** `vercel.json:95` rewrites non-`/api` paths to `/index.html`.
- **Indexing:** `X-Robots-Tag: noindex, nofollow, noarchive, nosnippet` (`vercel.json:8`).
- **Dev parity:** `vite.config.ts:5` mirrors prod headers/rate-limit/CORS in local middleware.

3.2 Supabase (Data Plane)

- **Engine:** Managed Postgres with `pgcrypto` for `gen_random_uuid()` (`supabase/schema.sql:3`).
- **Tables (8):** `users`, `cabins`, `bookings`, `locks`, `settings`, `booking_attendees`, `notifications`, `audit_logs` (`supabase/schema.sql:5`).
- **Indexes:** Cabin/date, user email, expires_at, notifications user+time, audit created/actor/action (`supabase/schema.sql:89`).
- **RLS:** Enabled on **all** tables (`supabase/schema.sql:99`). Explicit `DENY` policies `USING (false)` for role `anon` on every table (`supabase/schema.sql:108`) — if `anon` key leaks, zero rows leak. `service_role` (server) bypasses RLS by design; anonymous/public access is impossible.
- **Credentials:** `SUPABASE_URL` + `SUPABASE_SERVICE_ROLE_KEY` are **server-only env vars on Vercel** (`frontend/.env.example:5` notes `never VITE_`). The browser never receives them (`DATA.md:165`). Client SDK is created server-side with `persistSession:false, autoRefreshToken:false` (`lib/bookingApi.js:12`).

3.3 Secrets & Env Vars

Var	Exposure	Purpose	Where set
`VITE_GOOGLE_CLIENT_ID`	Public (bundled in JS, `src/App.tsx:12`) — this is an OAuth **client ID**, not a secret	Google OAuth provider init	Vercel env (build-time)
`GOOGLE_CLIENT_ID` / `GOOGLE_ALLOWED_CLIENT_IDS`	Server-only	Token audience (`aud`/`azp`) validation (`lib/bookingApi.js:715`)	Vercel env
`SUPABASE_URL`	Server-only	DB endpoint	Vercel env
`SUPABASE_SERVICE_ROLE_KEY`	Server-only, never `VITE_`	DB privileged access	Vercel env
`SLACK_BOT_TOKEN` (`xoxb-…`)	Server-only	Slack `users.lookupByEmail`, DM	Vercel env
`SLACK_SIGNING_SECRET`	Server-only	HMAC verify of Slack events (`lib/bookingApi.js:531`)	Vercel env

No secrets are committed. `.env.example` contains placeholders (`frontend/.env.example:1`). Rotating any secret only requires updating Vercel env + redeploy (no code change).

Authentication & User Lifecycle

4.1 How Users Log In

1. **Google OAuth (Authorization Code + access_token via GIS):** Clicking `Continue with Google` triggers `useGoogleLogin` with `hosted_domain: 'codeace.com'` — Google's picker is pre-filtered to that domain (`src/App.tsx:32`). Allowed domain constant is `codeace.com` (`lib/bookingApi.js:3`).
2. **Token verification (server, fail-closed):** `verifyGoogleToken` (`lib/bookingApi.js:712`) does:

- `GET https://oauth2.googleapis.com/tokeninfo?access_token=…` → extracts `aud`, `azp`, `hd`, `email`, `email_verified`.
- Validates **audience**: `aud` or `azp` must be in `GOOGLE_CLIENT_ID` / `GOOGLE_ALLOWED_CLIENT_IDS` allowlist — prevents cross-app token replay. Logs `aud mismatch` and returns `null` (`lib/bookingApi.js:757`).
- Validates **hosted domain**: `hd` must be `codeace.com` or empty (then email domain checked) (`lib/bookingApi.js:748`).
- Validates **email_verified** = true and email ends with `@codeace.com`.
- If `tokeninfo` is unreachable (`null`), **fails closed** — does not fall back to `userinfo` alone (`lib/bookingApi.js:769` "tokeninfo unavailable — failing closed").
- Fallback to `GET https://www.googleapis.com/oauth2/v3/userinfo` (Bearer) only when `tokeninfo` succeeded but lacked `email_verified/hd` — and re-checks `hd`/`email_verified` there (`lib/bookingApi.js:772`).
- Strips internal `_audOk` flag before returning `{ email, name }` (`lib/bookingApi.js:793`).

1. **Registration check:** `getRegisteredUser` (`lib/bookingApi.js:800`) queries `users` where `email = profile.email`. If missing → `USER_NOT_FOUND` ("Ask an admin to add your email first"). If `active=false` → `USER_INACTIVE`. If `name` drifted, silently syncs from Google profile.
2. **Session establishment:** Server sets `__Host-session` HttpOnly cookie (`lib/bookingApi.js:957`) on every authenticated response; client also keeps token in `sessionStorage` with 55-min TTL (`src/api/appsScript.ts:65`) to survive reloads within the same tab. Token is also accepted via `Authorization: Bearer` header (`lib/bookingApi.js:975`) and cookie as fallbacks (`lib/bookingApi.js:969`).
3. **Login audit:** `recordLogin` action writes `USER_LOGIN` to `audit_logs` (`lib/bookingApi.js:1079`, `src/App.tsx:51`).

A valid Google token **alone is not sufficient** — the email must exist and be active in the `users` table.

4.2 Logout & Session Revocation

- Client calls `recordLogout` (`src/api/appsScript.ts:336` → `lib/bookingApi.js:1091`) which writes `USER_LOGOUT` audit, then `revokeGoogleToken` (`lib/bookingApi.js:990` — `POST https://oauth2.googleapis.com/revoke?token=…`) so token replay fails, and `clearSessionCookie` (`lib/bookingApi.js:964` — `Max-Age=0`).

- Client also attempts `fetch(.../revoke)` best-effort (`src/App.tsx:78`), then `googleLogout()`, clears `sessionStorage` + legacy `localStorage` purge (`src/api/appsScript.ts:82`), nulls in-memory token.
- Cookie `Max-Age` is 55 min (`lib/bookingApi.js:944`), matching Google 60-min expiry minus buffer. No refresh token is stored anywhere (`DATA.md:18`).
- Remaining risk noted in `DATA.md:18`: token is still mirrored in `sessionStorage` (readable by XSS) until full HttpOnly-only migration (roadmap P0). `service_role` key never touches the browser.

4.3 Authorization (RBAC)

Role	Capabilities	Enforcement
`EMPLOYEE`	View cabins, Availability for a date, Own bookings + invited bookings, Notifications	Server checks pass — read actions allow any active user
`TEAM_LEAD`	Above + `createLock`, `refreshLock`, `confirmBooking`, `cancelLock`, cancel own future bookings	`validateBookingDateTime` + cabin `ACTIVE` check; `bookings` owner check (`lib/bookingApi.js:1493`)
`ADMIN`	All above + `allBookings`, `allUsers`, `create/update/delete` cabins & users, `importSlackUsers`, `updateUserStatus/Role`, `updateSettings`, `auditLogs`, cancel any booking	`requireAdmin(user)` guard on every admin action (`lib/bookingApi.js:873`, e.g. `1204`, `1524`, `1692`)

Additional RBAC rules:

- `updateUserRole` blocks self-role change and enforces **last-admin protection** (refuses demotion if only 1 active ADMIN remains, `lib/bookingApi.js:1838`).
- `cancelBooking` — non-admins cannot cancel past bookings (`lib/bookingApi.js:1498`), can only cancel their own.
- `deleteCabin` — blocked if future `BOOKED` bookings exist (`lib/bookingApi.js:1638`).
- Frontend `ProtectedRoute` (`src/App.tsx:164` + `src/components/ProtectedRoute.tsx`) is **UX only**; every action is re-checked server-side.

Data Inventory — What Is Collected, How It Is Stored

5.1 Users (`supabase/schema.sql:5` — `users` table)

Field	Required	Source	Example	
`email` (PK)	Yes	Google OAuth `email` (lower-cased)	`a@codeace.com`	
`name`	Yes	Google profile or admin/Slack/CSV	`Asha Rao`	
`employee_id`	No	Admin CSV/manual	`EMP042`	
`department`	No	Admin/CSV	`Engineering`	
`role`	Yes	`ADMIN \`	TEAM_LEAD \	EMPLOYEE`
`active`	Yes	Admin toggle	`true`	
`created_at`	Yes	`now()`	`2026-09-22T…`	

Created via: Admin form `createUser` (`lib/bookingApi.js:1692`), bulk `createUsers` (≤500, `lib/bookingApi.js:1722`), Slack import `importSlackUsers` (`lib/bookingApi.js:1764` — skips bots/guests/deleted), or Slack `team_join` event auto-create (`lib/bookingApi.js:686`).

5.2 Cabins (`supabase/schema.sql:15`)

`cabin_id` PK, `cabin_name`, `location`, `capacity` (1–50), `description`, `status` (`ACTIVE|INACTIVE`), `created_at`.

5.3 Bookings (supabase/schema.sql:25)

Field	Notes	
booking_id PK (BOOKING_ + uuid)	lib/bookingApi.js:266	
cabin_id FK, cabin_name (denormalized)		
date (date), start_time/end_time (text HH:MM)	30-min slots	
booked_by, booked_by_email FK→users, department	Cached from user row	
purpose (free text)	Meeting reason typed by organizer	
status (`BOOKED\	CANCELLED\	COMPLETED
created_at, updated_at		

Cancelled bookings are soft-retained, surfaced in Admin → Bookings and CSV export. Export columns: booking id, cabin, date, times, organizer name/email, department, purpose, member names, status, created/updated (DATA.md:72).

5.4 Booking Attendees / Members (supabase/schema.sql:59)

(booking_id FK cascade, user_email FK, name) — invitees added by email search (companyUsers action, lib/bookingApi.js:1218 — only active users, ordered by name). Attendee emails are validated: lower-cased, deduped, must end with @codeace.com, must be active users (lib/bookingApi.js:1419).

5.5 Temporary Locks (supabase/schema.sql:41)

Field	Notes
lock_id PK (LOCK_ + uuid)	supabase/schema.sql:196
cabin_id, date, start_time, end_time	Held slot
user_email FK	Owner
created_at, expires_at	Default + lock_duration_minutes (default 5)

Holds are deleted on confirm/cancel/expiry. Expired rows are pruned on availability queries (lib/bookingApi.js:1122).

5.6 Settings (supabase/schema.sql:52)

Single-row table id=1 CHECK (id=1) with lock_duration_minutes 1–30, max_booking_duration_minutes 15–240, advance_booking_days 1–90. Returned by settings action (lib/bookingApi.js:1269).

5.7 Notifications (supabase/schema.sql:67)

id uuid, user_email FK, booking_id, type (BOOKING_INVITE|BOOKING_CANCELLED), title, message, read, created_at. Created on booking/cancel (lib/bookingApi.js:1445, 1520). Read via notifications/markNotificationsRead (scoped to user_email = current user, lib/bookingApi.js:1236).

5.8 Audit Logs (supabase/schema.sql:78)

id uuid, created_at, actor_email, actor_name, action, entity_type, entity_id, summary. Indexed on created/actor/action (supabase/schema.sql:95). Retention 7 days, auto-pruned (lib/bookingApi.js:238 + 244). Admin-only read via auditLogs action (max 5000, last 7 days, sanitized search param — %*,() stripped, 80-char cap, lib/bookingApi.js:1890). Actions logged: USER_LOGIN, USER_LOGOUT, BOOKING_CREATED, BOOKING_CANCELLED, CABIN_CREATED/UPDATED/DELETED, USER_CREATED, USERS_IMPORTED, USER_ACTIVATED/DEACTIVATED, USER_ROLE_CHANGED, SETTINGS_UPDATED (lib/bookingApi.js:258).

5.9 Browser Storage

Item	Location	Lifetime	Notes
Google `access_token`	In-memory + `sessionStorage` (`TOKEN_SESSION_KEY`)	55 min TTL, cleared on tab close	`src/api/appsScript.ts:20`, `DATA.md:120`
`__Host-session` cookie	HttpOnly, Secure, SameSite=Lax, Path=/	55 min	`lib/bookingApi.js:957`
No booking data persisted client-side	—	—	All state in Postgres

Legacy `localStorage` key `google_access_token` is purged on load (`src/api/appsScript.ts:82`).

5.10 What Is Not Collected

Passwords, phone numbers, home addresses, payment data, calendar contents (beyond bookings created here), files/photos, GPS/location, advertising cookies (`DATA.md:127`). Google refresh tokens are never stored.

Security Precautions & Controls

6.1 Network & Transport

- **HTTPS only** — Vercel auto-TLS + HSTS preload (`vercel.json:28`, `lib/bookingApi.js:983`). `Cache-Control: no-store` on API (`vercel.json:68`, `lib/bookingApi.js:984`).
- **CORS — strict allowlist** (`lib/bookingApi.js:1001`): exact host or subdomain of `cabin.codeace.org` / `cabin.codeace.com` over `https:` only; `origin` must parse as URL with `protocol === 'https:'`. Prevents `evilcodeace.org` suffix bypass. `Vary: Origin` set.
- **No public API** — every request requires valid Google token + registered user (`lib/bookingApi.js:1049`).

6.2 Authentication Hardening

- Domain restriction `codeace.com` at three layers: GIS `hosted_domain` (`src/App.tsx:32`), server `ALLOWED_DOMAIN` const (`lib/bookingApi.js:3`), and `hd` check in `verifyGoogleToken` (`lib/bookingApi.js:748`).
- Audience binding (`aud/azp` vs `GOOGLE_CLIENT_ID`/`GOOGLE_ALLOWED_CLIENT_IDS`) prevents token replay across OAuth clients (`lib/bookingApi.js:715`).
- `email_verified` required (`lib/bookingApi.js:749`, `782`).
- Fail-closed on `tokeninfo` outage (`lib/bookingApi.js:769`).
- Token revocation on logout (`lib/bookingApi.js:990` + `src/App.tsx:78`).

6.3 Anti-CSRF

- API requires `Content-Type: application/json` (`lib/bookingApi.js:1030`). `<form>`-based CSRF cannot set this without a preflight, which the CORS check would block.
- Client uses `fetch` with `Content-Type: application/json` + `credentials:'include'` (`src/api/appsScript.ts:149`) and server validates JSON response (`src/api/appsScript.ts:158`).

6.4 Rate Limiting

In-memory per-IP (60 req/min) and per-user (30 req/min) with `Retry-After` header (`lib/bookingApi.js:920` — "pair with edge KV in prod" noted for Vercel distribution). Keyed by `x-forwarded-for` / `x-real-ip` (`lib/bookingApi.js:939`).

6.5 Input Validation & Injection Defense

- `normalizeUserInput` (`lib/bookingApi.js:890`) trims + lower-cases email, derives name from email local-part if missing (`lib/bookingApi.js:879`), validates role against allowlist (`VALID_ROLES`, `lib/bookingApi.js:877`), enforces `@codeace.com`.
- Booking validation `validateBookingDateTime` (`lib/bookingApi.js:844`): rejects past, beyond `advanceBookingDays`, zero/negative duration, exceeds `maxBookingDurationMinutes`.
- SQL injection: All DB access via Supabase client parameterized queries / `rpc` (`lib/bookingApi.js:6`). No string-concatenated SQL except `times_overlap` helper (immutable SQL function, `supabase/schema.sql:145`).
- Slack search sanitization: `%*,()` stripped, 80-char slice (`lib/bookingApi.js:1892`).

- Bulk import capped at 500 rows, 100-row chunks (`lib/bookingApi.js:1731`, `632`).

6.6 Concurrency & Data Integrity

- **Advisory lock** `pg_advisory_xact_lock(hashtext(cabin:date))` serializes all lock/create/confirm paths per cabin+date inside a single Postgres transaction (`supabase/schema.sql:170`, `234`). Eliminates race/double-book without table-level locking.
- Overlap detection via `times_overlap` (`start1 < end2 AND end1 > start2`, `supabase/schema.sql:145`) and JS `doTimesOverlap` (`lib/bookingApi.js:74`) — applied to both bookings and active locks.
- Functions `create_booking_lock` and `confirm_booking_from_lock` are `REVOKE`d from `public`, `GRANT EXECUTE` only to `service_role` (`supabase/schema.sql:299`).
- Timezone is `Asia/Kolkata` everywhere (`lib/bookingApi.js:4`, `supabase/schema.sql` date handling).

6.7 XSS & Clickjacking

- **CSP (global,** `vercel.json:32`**):** `default-src 'self'; script-src 'self' https://accounts.google.com https://www.gstatic.com; connect-src 'self' https://accounts.google.com https://www.googleapis.com https://oauth2.googleapis.com; img-src 'self' data: https:; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src ...; frame-src https://accounts.google.com; frame-ancestors 'none'; base-uri 'self'; form-action 'self'` — tight allowlist for Google Identity only.
- **Clickjacking:** `X-Frame-Options: DENY` + `frame-ancestors 'none'` + `Cross-Origin-Opener-Policy: same-origin-allow-popups` (`vercel.json:15/35`).
- **MIME sniffing:** `X-Content-Type-Options: nosniff`.
- **Permissions:** `Permissions-Policy: camera=(), microphone=(), geolocation=()`.
- **Isolation:** `Cross-Origin-Embedder-Policy: credentialless`, `Cross-Origin-Resource-Policy: same-origin`.
- **Error handling:** Generic user messages — infra strings (`supabase`/`vercel`) are stripped (`src/api/appsScript.ts:131` — "no infra details leaked").

6.8 Authorization & RLS

- Server-side RBAC on every action (see §4.3).
- Postgres RLS deny-by-default for `anon` (see §3.2). Direct Supabase `anon` key cannot read any row even if leaked.
- Sensitive actions write `audit_logs` with actor + summary.

6.9 Secrets & Supply Chain

- Only `VITE_GOOGLE_CLIENT_ID` is client-exposed; all other secrets are server env vars (`frontend/.env.example:3` comment "Server-only (never VITE_)").
- No credentials in frontend code (`src/App.tsx:12` only reads `VITE_GOOGLE_CLIENT_ID`).
- Dependencies: `react`, `react-router-dom`, `@react-oauth/google`, `@supabase/supabase-js` (`frontend/package.json:12`). Lockfile committed (`package-lock.json`). Build uses `tsc -b && vite build` (`package.json:8`).

6.10 Logging, Monitoring & Retention

- **Audit logs:** 7-day retention, pruned on every write + on `auditLogs` read (`lib/bookingApi.js:244`, `1895`). Max 5000 rows returned. Only admins can read.
- **Notifications:** Per-user scoped, limit 50 on read (`lib/bookingApi.js:1242`).
- **Server errors:** Caught and returned as generic `SERVER_ERROR` ("Something went wrong…") — stack not leaked (`lib/bookingApi.js:1921`).
- **Client errors:** Generic "Service temporarily unavailable" on JSON parse failure (`src/api/appsScript.ts:164`).

6.11 Slack Integration Security

- `x-slack-request-timestamp` age ≤ 5 min enforced (`lib/bookingApi.js:542` — "60*5").

- `x-slack-signature` verified as `v0=HMAC-SHA256(secret, v0:timestamp:rawBody)` with constant-time `safeEqual` (`lib/bookingApi.js:508`, `518`).
- `/api/slack/events` also supports `url_verification` challenge (`lib/bookingApi.js:683`).

6.12 OWASP Top-10 Mapping

OWASP	Control in this system
A01 Broken Access Control	RBAC server-side + RLS `USING(false)` for anon + last-admin guard
A02 Cryptographic Failures	HSTS preload, HTTPS only, no secrets in repo, HMAC for Slack
A03 Injection	Parameterized Supabase queries, sanitized audit search, RPC-only writes
A04 Insecure Design	Advisory locks + overlap checks + fail-closed token verify
A05 Security Misconfiguration	Hardened Vercel headers, CSP, no directory listing, env allowlist
A06 Vulnerable Components	Pinned deps + lockfile, minimal surface (no calendar/email legacy code in live path)
A07 Auth Failures	Google OAuth aud/hd/email_verified, active-user check, token revoke on logout
A08 Data Integrity	Signed Slack webhooks, transaction-scoped advisory locks
A09 Logging Failures	Audit log per state-changing action, 7-day retention, generic error messages
A10 SSRF	No user-controlled fetch targets; `SLACK_BOT_TOKEN` only calls `slack.com` allowlisted hosts

Privacy & Compliance

- **Lawful basis:** Employment/contract + legitimate interest for room management.
- **Data minimization:** Only work email + name + optional department/employee ID (`DATA.md:26`). No passwords, phone, address, payment, files, GPS (`DATA.md:127`).
- **Access scope:** Employees/Team Leads see own + invited bookings; admins see all (see §5.10, `DATA.md:138`). `companyUsers` returns only active users' email/name/dept/role.
- **Retention:** Bookings (including `CANCELLED`) retained for admin export; locks are ephemeral; audit 7 days (`lib/bookingApi.js:238`); notifications per-user until read. No automated deletion of bookings — manual via `deleteCabin` cascade (`lib/bookingApi.js:1669`) or DB purge.
- **Subject rights:** Admin can deactivate (`updateUserStatus`, `lib/bookingApi.js:1781`) or change role; deletion requires DB `DELETE FROM users WHERE email=…` (cascade via `booking_attendees` FK). Export is via `allBookings` + CSV in Admin UI.
- **Cross-border:** Data stored in Supabase Cloud region (configure to required residency). No third-party analytics/advertising cookies beyond hosting (`DATA.md:135`).
- **Slack data:** Only email + name copied; DM text contains cabin/date/time/purpose/leader (`lib/bookingApi.js:373` via `slackEscape`). No booking details posted to channels.

Known Limitations & Roadmap (Security-Relevant)

Item	Current	Planned
Session token storage	`sessionStorage` mirror + HttpOnly `__Host-session` (`src/api/appsScript.ts:20`, `lib/bookingApi.js:943`) — XSS could still read `sessionStorage` copy	**P0:** HttpOnly-only auth code flow, remove token from body (`src/api/appsScript.ts:17` comment)
Rate limit distribution	In-memory per instance (`lib/bookingApi.js:920` "pair with edge KV in prod") — Vercel scales to multiple instances, limit is per-instance	Edge KV / Upstash Redis
Token lifetime	55 min TTL, refreshed via re-login only (no silent refresh)	Silent refresh via HttpOnly session

Operational Checklist for Compliance Review

- [ ] Verify Vercel env vars: `GOOGLE_CLIENT_ID`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SLACK_BOT_TOKEN`, `SLACK_SIGNING_SECRET` are set and **not** prefixed `VITE_`.
- [ ] Confirm Supabase RLS: `SELECT * FROM pg_policies WHERE schemaname='public'` shows 8 `deny anon` policies + `enable row level security` on all 8 tables.
- [ ] Confirm `vercel.json` headers deployed (check response headers on `https://cabin.codeace.org` and `/api`).
- [ ] Test auth negative cases: non-`codeace.com` email, inactive user, token with wrong `aud` — all must return `UNAUTHORIZED` / `USER_NOT_FOUND` without leaking stack.
- [ ] Test RBAC: EMPLOYEE cannot hit `createLock`/`allBookings`; non-admin cannot `updateUserRole`.
- [ ] Test concurrency: two `TEAM_LEAD`s creating lock for same slot — one must receive `SLOT_BOOKED`/`SLOT_LOCKED` (`supabase/schema.sql:182`, `195`).
- [ ] Review `audit_logs` retention: `SELECT COUNT(*) FROM audit_logs WHERE created_at < now() - interval '7 days'` should be 0 after a write.
- [ ] Rotate `SUPABASE_SERVICE_ROLE_KEY` and `SLACK_*` on a schedule; revoke old Google OAuth client if `GOOGLE_CLIENT_ID` changes.

References (Code Anchors)

- `frontend/vercel.json:1` — security headers, CSP, HSTS, rewrites
- `frontend/src/App.tsx:12` — OAuth entry, `hosted_domain` restriction
- `frontend/src/api/appsScript.ts:1` — client API, 55-min TTL, generic error mapping, `credentials:'include'`
- `frontend/lib/bookingApi.js:1` — server handler, Supabase client, all validations, audit, Slack, RBAC, rate limit, token verify
- `frontend/lib/bookingApi.js:712` — `verifyGoogleToken` (aud/azp/hd/email_verified, fail-closed)
- `frontend/lib/bookingApi.js:998` — `handleBookingApi` (CORS allowlist, CSRF, rate limit, auth gate)
- `supabase/schema.sql:1` — canonical schema, RLS, functions, grants
- `frontend/api/index.js:1` — Vercel function entry (Node)
- `frontend/api/slack/events.js:1` — Slack events (Edge) + `lib/bookingApi.js:531` signature verify
- `DATA.md:1` — data inventory (source of truth for PII scope)
- `frontend/.env.example:1` — required env vars (server-only vs `VITE_` boundary)

Contact & Ownership

Role	Responsibility
App owner	Admins (`users` where `role='ADMIN'`) — user/cabin/settings management
Hosting	Vercel (app) + Supabase (DB) — infra patching managed by vendors
Auth IdP	Google Workspace (`codeace.com`) — account lifecycle source of truth
Security contact	*(fill: security@codeace.com / IT lead)*